25DCE026

Department of Computer Engineering

Practical - 13

Practical Title: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Problem Definition: Create PHP JSON API endpoints for students and consume them using the Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

Theory :

1. Do PHP endpoints return valid JSON responses?

Yes. All endpoints in `api/students.php` set `Content-Type: application/json` and return standardized JSON objects (`status`, `message`, `data`, `count`) alongside HTTP status codes (200, 201, 400, 404, 422).

2. Are GET and POST requests handled properly?

Yes. GET requests list and search records. POST bodies are parsed via `file_get_contents('php://input')` and decoded with `json_decode()` to handle asynchronous create, update, and delete actions.

3. Are loading, success, and error states shown on the frontend?

Yes. A CSS spinner indicates active network requests; toast notifications confirm success; error alerts surface validation or network failures.

4. How is AJAX search optimized with debounce?

A 300ms debounce timer resets with every keystroke and fires the Fetch API GET request only after typing pauses for 300 milliseconds, preventing excessive network calls.

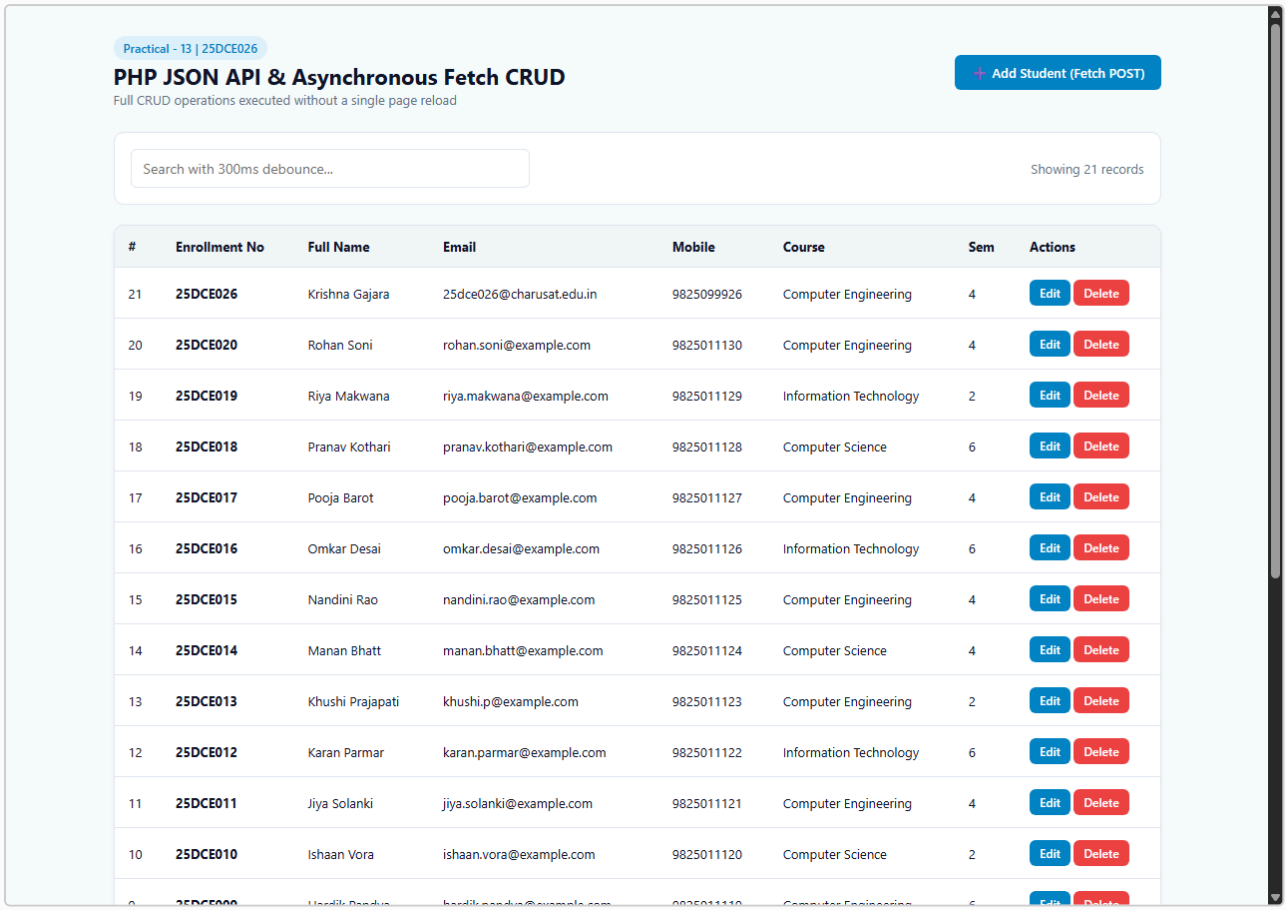
Implementation / Output / Result

```
// Asynchronous Fetch API POST without Page Reload
const response = await fetch('api/students.php', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ action: 'create', full_name, email, course, ... })
});
const res = await response.json();
if (res.status === 'success') showToast(res.message);
```

25DCE026

Output 1: Fetch API Single Page Application (SPA) with Live Debounced Search

Live browser screenshot: Full SPA with student table, debounced live search input, Add Student modal button — zero page reloads.



GitHub Repository Link:
https://github.com/25dce026-krishna/StudentHub

Learning Outcomes:

- Built RESTful JSON backend APIs with PHP and MySQLi.
- Mastered JavaScript Fetch API using `async/await` and Promises for asynchronous operations.
- Developed a Single Page Application (SPA) performing complete CRUD without browser refreshes.
- Managed asynchronous UI feedback states including loading spinners and toast notifications.
- Optimized client-to-server traffic using a 300ms search input debounce algorithm.
- Utilized browser developer network inspection tools to verify API request/response lifecycles.